

ECOCIVIX AI

10 — Hackathon Execution Plan

TEAM VELOCITY • HackYatra × EDM 3.0 • PRE-HACKATHON PLANNING

Purpose: Provide the team with a practical operating plan for the 24-hour onsite hackathon while preserving flexibility for the official problem statement.

Status: PRE-HACKATHON DRAFT / PLANNING SPECIFICATION

Approval authority: Garv Nain

Before the Start

- Read the official problem statement and rules first.
- Compare ECOCIVIX against the challenge.
- Identify what matches, changes, removals and additions.
- Garv makes the final product decision.

Operating Mode

- Build → Integrate → Test → Polish → Freeze.
- If core integration breaks, stop feature development and fix integration first.

Phase 1 — Foundation

- Repository and branch setup; environment variables; Supabase project; authentication; base frontend/backend structure; agreed API/DB contracts.

Phase 2 — Core Development

- Garv: AI/orchestration/integration foundation. Rajveer: backend/database/auth/APIs. Gunjan: citizen/admin UI and responsive implementation. Lokesh: test harness, integration checklist and deployment preparation.

Phase 3 — Integration

- Connect frontend to real APIs; connect APIs to Supabase; connect AI; verify auth and issue lifecycle; run end-to-end report flow.

Phase 4 — Testing & Polish

- Hard testing, error states, security checks, responsive fixes, performance fixes, visual polish and one strong wow feature only if MVP is stable.

Phase 5 — Deployment & Demo

- Deploy production build; verify environment variables; run full demo path; freeze features; rehearse; prepare disaster fallback.

Suggested Time Blocks

- 0:00–0:45 problem/rules; 0:45–2:00 architecture/setup; 2:00–6:00 core parallel build; 6:00–8:00 first integration; 8:00–12:00 MVP; 12:00–14:00 hard testing; 14:00–17:00 polish; 17:00–19:00 wow feature; 19:00–20:00 deployment; 20:00–22:00 final QA/presentation; 22:00–23:30 rehearsal/disaster prep; 23:30–24:00 feature freeze.

Git Rules

- Branch-based work; no force push; no history rewriting; focused commits; no modifying another member's ownership without approval; verify before integration; Garv controls final integration.

Communication

- Report blockers with objective, failure, exact error, inspected files/logs, suspected missing piece and decision required.

Feature Triage

- Critical functionality → integration → reliability → security → testing → UX → performance → visual polish → optional features.

Completion Gate

- A feature is complete only after implementation, tests/checks, integration verification and a clear completion report.

Disaster Preparation

- Keep a known-good deployment/build; retain rollback-capable commits; prepare static fallback for advanced visuals; maintain a final demo dataset generated through the real application flow.